

Block Round

Les Mathématiques du Projet

Documentation technique et didactique des fondements mathématiques employés dans le générateur de cercles, ellipses, sphères et ellipsoïdes *rendus avec des textures de blocs Minecraft*. Pour chaque concept, sont présentés la définition formelle, la justification géométrique, la correspondance directe avec le code source du projet et la traduction pratique en commandes de construction in-game (`/fill`, `/clone`, `//sphere`).

Domaines abordés : géométrie analytique · rasterisation discrète · topologie digitale · calcul intégral · algèbre linéaire · trigonométrie · arithmétique d'inventaire · mappage de textures · symétrie octaédrique.

Table des matières

1	Vue d'ensemble : nature mathématique du problème	3
2	Système de coordonnées et grille discrète de voxels	4
3	L'équation fondamentale : cercle et ellipse	4
4	Algorithme Euclidien (test de distance)	5
5	Algorithme de Bresenham (point milieu entier)	6
5.1	Cas circulaire	6
5.2	Cas elliptique (deux régions)	7
6	Algorithme de Seuil (couverture de coin)	7
7	Modes de rendu : Plein, Mince, Épais	8
7.1	Plein	8
7.2	Mince (contour)	8
7.3	Épais	9
8	Calcul d'aire discrète	9
9	De 2D à 3D : l'équation de l'ellipsoïde	10
10	Voxelisation et calcul de volume	10
11	Coupes géométriques : plans et coupe diagonale à 45°	11
12	Mode Épais 3D : tridimensionnalisation par tranches	12
13	Caméra 3D : coordonnées sphériques	12
14	Auto-zoom : sphère englobante et FOV	13
15	Ombrage et multiplication de texture	13
16	Transition : des mathématiques continues à la construction in-game	14
17	Arithmétique de l'inventaire Minecraft	15
17.1	Emplacement unique, pile unique, pack plein	15
17.2	Stockage : coffres, coffres doubles, boîtes de shulker	15
17.3	Tableau de référence : sphères canoniques	15

18 Surlignage de contour et comptage des faces exposées	16
18.1 Qu'est-ce qu'une face exposée	16
18.2 Formule de détection de frontière	16
18.3 Comptage de faces exposées comme heuristique de survie	16
19 Construction couche-par-couche (décomposition horizontale)	17
20 Optimisation par symétrie octaédrique (O_h)	18
20.1 Séquence de commandes concrète	18
20.2 Comparaison de temps	18
21 Textures de blocs et composition visuelle	19
21.1 Les six faces et le mappage UV	19
21.2 Familles de blocs et tons	19
21.3 Dégradés par mélange de blocs	20
22 Conclusion	20
Annexe A : tableaux de référence de construction et exemple détaillé	22

1. Vue d'ensemble : nature mathématique du problème

Block Round est un générateur de formes arrondies *texturées avec des blocs Minecraft* : cercles et ellipses en 2D, sphères et ellipsoïdes en 3D. Le problème central appartient à la *rastérisation discrète* : étant donné une courbe ou une surface définie analytiquement sur $\mathbb{R}$, déterminer quelles cellules d'une grille régulière (pixels en 2D, voxels en 3D) doivent être marquées pour la représenter. Chaque cellule marquée dans Block Round reçoit en outre une texture à six faces du catalogue Minecraft (Bloc d'Herbe, Pierres Moussues, Planches de Chêne, Quartz, Diamant, etc.), de sorte que la figure résultante n'est pas seulement une silhouette mathématique mais une *structure constructible* que l'utilisateur peut reproduire en survie, en créatif avec `/fill`, ou importer via Sponge Schematic v2 (`.schem`) dans WorldEdit, Litematica ou MCEdit.

Le problème n'admet pas de solution unique. Différents critères d'inclusion produisent différentes silhouettes discrètes, chacun avec sa propre justification mathématique. Le projet implémente donc trois algorithmes 2D distincts (Euclidien, Bresenham, Seuil), trois modes de rendu (Plein, Fin, Épais) et trois styles d'ombrage 3D. Le choix de l'algorithme dépend non seulement de la douceur visuelle mais aussi du nombre de blocs à collecter — une sphère de diamètre $D = 20$ requiert 4 189 blocs avec Euclidien mais jusqu'à 4 322 avec Seuil.

L'exécution se fait entièrement dans le navigateur, sans composant serveur, ce qui impose un *rendement de calcul* (une sphère de Diamant de $D = 32$ doit rendre à 60 fps avec 17 156 voxels texturés). Les domaines théoriques mobilisés incluent :

- géométrie analytique des coniques et quadriques ;
- algorithmes incrémentaux à arithmétique entière (Bresenham) ;
- topologie digitale (voisinages 4-, 6- et 26-connexes) ;
- calcul intégral et mesure de comptage ;
- algèbre linéaire (plans de coupe, projection perspective) ;
- trigonométrie et coordonnées sphériques ;
- arithmétique combinatoire d'inventaire et symétrie de groupes.

Nouveautés par rapport au projet frère Pixel Round. Block Round ajoute cinq préoccupations mathématiques nouvelles propres au fait que les cellules rendues deviennent des *blocs Minecraft* : (i) le mappage UV des six faces de chaque

voxel; (ii) l'arithmétique de l'inventaire de survie (packs de 64 items, coffres simples de 27 emplacements, coffres doubles de 54 emplacements); (iii) le sur-lignage de contour utilisé pour compter les faces exposées pendant la construction; (iv) la décomposition d'une forme 3D en couches horizontales, puisque la construction réelle procède étage par étage; (v) l'exploitation du groupe de symétrie octaédrique O_h pour réduire le coût de pose manuelle d'un facteur 8 via `/clone`.

2. Système de coordonnées et grille discrète de voxels

Le canevas est une grille de $G_x \times G_y$ cellules en 2D, étendue à $G_x \times G_y \times G_z$ voxels en 3D. Chaque cellule (i, j) occupe le carré unité $[i, i+1] \times [j, j+1]$; en 3D chaque voxel occupe le cube unité. Dans le code continu, le **centre** de la cellule est en $(i + \frac{1}{2}, j + \frac{1}{2})$. Cette convention est critique : comparer la circonférence au *coin* (i, j) ou au *centre* change quelles cellules appartiennent à la forme et donc quels blocs l'utilisateur doit poser.

$$\text{centre de la cellule } (i, j) = (i + \frac{1}{2}, j + \frac{1}{2})$$

Pour une forme de diamètre D , le code étend la grille à $D + 2$ cellules par axe (marge d'une cellule de chaque côté). Cela garantit que les pixels extrêmes ne tombent jamais sur le bord de la matrice. Le **centre** de la forme est en $(G_x/2, G_y/2)$, et les *demi-axes* sont $r_x = D/2$ et $r_y = D/2$. La cellule unité du canevas coïncide avec le bloc unité de Minecraft : les coordonnées du code se mappent un-à-un sur les coordonnées (x, y, z) du jeu.

Mapping Minecraft. Si le joueur apparaît en $(0, 64, 0)$ et veut une sphère $D = 16$ ($r = 8$) posée au sol, le centre doit être en $(0, 72, 0)$ (8 blocs au-dessus de la surface), pour que le pôle sud touche le sol. Commande WorldEdit : `//sphere stone 8` depuis $(0, 72, 0)$.

3. L'équation fondamentale : cercle et ellipse

Toute la théorie du projet part de l'équation implicite de l'ellipse centrée à l'origine avec demi-axes r_x et r_y :

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 = 1$$

Quand $r_x = r_y = r$, l'ellipse dégénère en une **circonférence** de rayon r . Les points de l'*intérieur* satisfont ≤ 1 ; ceux de l'*extérieur*, > 1 . C'est l'inégalité qui définit si un voxel appartient à la forme — et donc si l'utilisateur pose un bloc à cet endroit.

En déplaçant le centre vers (c_x, c_y) et en évaluant au centre de chaque cellule :

$$d_x = \frac{i + \frac{1}{2} - c_x}{r_x}, \quad d_y = \frac{j + \frac{1}{2} - c_y}{r_y}, \quad \text{dedans} \iff d_x^2 + d_y^2 \leq 1$$

Mapping Minecraft. Pour une tour en survie, on veut typiquement un *cercle aplati large* pour les fondations et des cercles plus petits à mesure que la tour monte. Chaque étage est une application de l'équation implicite de l'ellipse avec les mêmes demi-axes et un z différent. Block Round calcule la couche 2D une fois, l'utilisateur duplique avec `/clone` par étage.

4. Algorithme Euclidien (test de distance)

Idée. Pour chaque ligne j , trouver analytiquement les colonnes i dans l'ellipse. En résolvant l'équation pour x donné y :

$$x = c_x \pm r_x \sqrt{1 - \left(\frac{y - c_y}{r_y}\right)^2}$$

Pour une ligne j de déplacement vertical $d_y = j + \frac{1}{2} - c_y$, la *demi-largeur horizontale* de l'ellipse à cette hauteur est :

$$dxM = r_x \sqrt{1 - \frac{d_y^2}{r_y^2}}$$

Si $d_y^2/r_y^2 \geq 1$, la ligne n'intersecte pas l'ellipse et est sautée. Sinon, toutes les colonnes i telles que $c_x - dxM \leq i + \frac{1}{2} \leq c_x + dxM$ sont remplies. Les bornes entières viennent de :

$$lo = \lceil c_x - dxM - \frac{1}{2} \rceil, \quad hi = \lfloor c_x + dxM - \frac{1}{2} \rfloor$$

Justification. Le terme $+\frac{1}{2}$ résulte de la convention selon laquelle la cellule i a son centre en $i + \frac{1}{2}$. Les fonctions $\lceil \cdot \rceil$ et $\lfloor \cdot \rfloor$ convertissent l'intervalle continu en indices entiers, garantissant que seules les cellules dont le **centre** appartient à l'intérieur de l'ellipse sont incluses. Cette formulation produit l'approximation discrète la plus proche de la courbe continue et donc la silhouette la plus lisse. Pour des petits diamètres, le résultat peut sembler moins pixel-art — un cercle Euclidien $D = 5$ n'a que 13 blocs, contre 21 pour Seuil.

Mapping Minecraft. Le critère Euclidien est le plus proche de ce que produit `//sphere` dans WorldEdit. Pour une sphère de Pierres Moussues $D = 16$ ($r = 8$) avec coque Euclidienne uniquement, $V \approx 2\,145$ blocs en plein, ≈ 804 en mince — la différence entre 34 packs (2 176) et 13 packs (832).

5. Algorithme de Bresenham (point milieu entier)

L'algorithme de Bresenham, publié en 1965 pour les segments, a été étendu aux cercles puis généralisé par Pitteway et Kappel aux ellipses arbitraires. Sa propriété distinctive est de se passer de virgule flottante et de racine carrée : la détermination du prochain pixel utilise exclusivement **additions et comparaisons entières**, via une *fonction de décision* évaluant de quel côté de la courbe analytique se trouve le milieu entre les deux pixels candidats. Pour la construction Minecraft, c'est l'algorithme qui livre le look pixel-art le plus *reconnaissable* — la même silhouette en escalier que la communauté construit à la main depuis 2011.

5.1 Cas circulaire

Pour un cercle de rayon entier R , on part de $(x=0, y=R)$ avec le paramètre de décision initial :

$$d_0 = 1 - R$$

À chaque étape (dans l'octant de 0 à 45°) :

$$\begin{cases} \text{si } d < 0 : & \text{le suivant est } (x+1, y), \quad d += 2x + 3 \\ \text{si } d \geq 0 : & \text{le suivant est } (x+1, y-1), \quad d += 2(x - y) + 5 \end{cases}$$

Justification. La fonction d approxime, à chaque itération, la valeur de l'équation

implicite du cercle évaluée au **point milieu** entre les deux pixels candidats :

$$F(x+1, y-\frac{1}{2}) = (x+1)^2 + (y-\frac{1}{2})^2 - R^2$$

Le signe de d indique si le milieu est dedans (avancer seulement en horizontal) ou dehors (décrémenter aussi y). Les huit octants s'obtiennent par les symétries du groupe diédral D_4 , soit en code les huit appels à `span(...)`. Le tracé présente le motif en escalier typique des approximations discrètes de courbes lisses.

5.2 Cas elliptique (deux régions)

Pour une ellipse de demi-axes a, b , l'algorithme divise le premier quadrant en **deux régions** délimitées par le point où la tangente fait 45° :

$$\text{Région I : } 2b^2x < 2a^2y \quad (|dy/dx| < 1), \quad \text{Région II : sinon}$$

Les paramètres de décision initiaux sont :

$$\begin{aligned} p_1 &= b^2 - a^2b + \frac{a^2}{4}, \\ p_2 &= b^2\left(x + \frac{1}{2}\right)^2 + a^2(y-1)^2 - a^2b^2. \end{aligned}$$

À chaque itération, p est mis à jour par des incréments pré-calculés (tous entiers après multiplication par 4). Le résultat est le minimum d'arithmétique par pixel : *un test de signe et deux additions*.

Mapping Minecraft. Pour les bâtisseurs en survie, la silhouette Bresenham est souvent préférable à l'Euclidienne car ses pas discrets sont faciles à compter manuellement — l'utilisateur sait que le prochain bloc est toujours *droit* ou *diagonal*. Un cercle Bresenham $D = 11$ a la séquence canonique de 80 blocs publiée dans des dizaines de guides.

6. Algorithme de Seuil (couverture de coin)

Cet algorithme demande : *existe-t-il un coin de cellule à l'intérieur de l'ellipse ?* Pour chaque cellule (i, j) , le code trouve le coin le plus proche du centre (projection du centre sur les arêtes) :

$$n_x = \text{clamp}(c_x, i, i+1), \quad n_y = \text{clamp}(c_y, j, j+1)$$

$$\text{dedans} \iff \left(\frac{n_x - c_x}{r_x} \right)^2 + \left(\frac{n_y - c_y}{r_y} \right)^2 \leq 0,94$$

La valeur 0,94 est un *seuil empirique* légèrement inférieur à 1,0. Son but est d'éliminer l'artefact appelé *pic tangentiel* de 1 pixel aux quatre points cardinaux, où la courbe est tangente à une arête entière de la cellule. Sans cette réduction, l'algorithme ajouterait une cellule supplémentaire qui n'améliore pas la représentation visuelle.

Parmi les trois algorithmes, c'est celui qui produit la silhouette de plus grande aire pour le même D , puisque la condition d'inclusion est la moins restrictive. Pour Minecraft, c'est l'algorithme qui donne le look le plus *arrondi* au prix de plus de blocs — $D = 16$ produit 213 blocs de coque sous Seuil, contre 200 sous Bresenham et 192 sous Euclidien.

Mapping Minecraft. Seuil est le choix quand la construction sera vue de loin (par exemple un dôme de Quartz sur un temple), car la silhouette légèrement plus grande se lit mieux à travers les ombres d'occlusion ambiante. Compromis : environ 5–10% de blocs supplémentaires par coque par rapport à Euclidien.

7. Modes de rendu : Plein, Mince, Épais

Avec une matrice $f[j][i] = 1$ pour les cellules intérieures, le projet dérive trois styles visuels par **topologie discrète** :

7.1 Plein

Retourne f inchangé. Toutes les cellules internes deviennent des blocs. C'est ce que produit `//sphere stone 8`.

7.2 Mince (contour)

Une cellule appartient au contour mince si elle est remplie *et* possède au moins un voisin orthogonal (N, S, E, O) *en dehors* de la forme. En notation logique :

$$b(i, j) = f(i, j) \wedge (\neg f(i-1, j) \vee \neg f(i+1, j) \vee \neg f(i, j-1) \vee \neg f(i, j+1))$$

Géométriquement : c'est la **frontière discrète** de la forme, analogue discret de la frontière topologique $\partial F = F \cap \overline{F^c}$. En Minecraft c'est l'équivalent de `//hsphere` :

seule la coque est matérialisée.

7.3 Épais

Le mode épais ajoute aux pixels de bord les **ponts diagonaux** : cellules internes ayant simultanément un voisin-bord horizontal et un voisin-bord vertical. Cela ferme les diagonales d'un pixel laissées ouvertes par le mode mince, utile quand le contour doit paraître stable dans une scène (sans trous à 45°) — particulièrement important pour les dômes en Verre et Glace.

$$\begin{aligned} t(i, j) &= b(i, j) \vee (f(i, j) \wedge h_H \wedge h_V), \\ h_H &= b(i-1, j) \vee b(i+1, j), \\ h_V &= b(i, j-1) \vee b(i, j+1). \end{aligned}$$

Mapping Minecraft. Pour un dôme de Verre $D = 20$ (creux), le mode épais donne une coque *étanche* de 608 blocs contre ≈ 510 en mince — les ~ 100 blocs supplémentaires sont la couture diagonale qui empêche la lumière ambiante de fuir.

8. Calcul d'aire discrète

La fonction `area2D` **compte** simplement les cellules marquées dans f . C'est la *mesure de comptage*, qui approxime l'intégrale de Riemann de la fonction indicatrice de l'ellipse :

$$\underbrace{\pi r_x r_y}_{\text{aire continue}} \quad \longleftrightarrow \quad \underbrace{\sum_{j=0}^{G_y-1} \sum_{i=0}^{G_x-1} f(i, j)}_{\text{aire discrète}}$$

Pour D grand, aire discrète/aire continue $\rightarrow 1$. Pour D petit, la différence entre algorithmes est visible : Seuil tend à surestimer ; Euclidien reste proche de l'aire idéale ; Bresenham se situe entre les deux.

Mapping Minecraft. La puce de comptage de blocs dans l'UI affiche exactement ce nombre : 268 pour $D = 8$, 904 pour $D = 12$, 2145 pour $D = 16$. La puce exprime le compte comme N (*pires* $\times 64$ + *reste*) — ainsi $2145 = 33 \times 64 + 33$, soit « 33 piles plus 33 items ».

9. De 2D à 3D : l'équation de l'ellipsoïde

En 3D, la forme fondamentale est l'**ellipsoïde** centré en (c_x, c_y, c_z) avec demi-axes r_x, r_y, r_z . Son équation généralise l'ellipse :

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 + \left(\frac{z}{r_z}\right)^2 = 1$$

Quand $r_x = r_y = r_z$, on retrouve la **sphère**. En évaluant au centre de chaque voxel :

$$a_\xi = \frac{\xi + \frac{1}{2} - c_\xi}{r_\xi} \quad (\xi \in \{x, y, z\}), \quad \text{dedans} \iff a_x^2 + a_y^2 + a_z^2 \leq 1$$

Mapping Minecraft. Un ellipsoïde $W = 20$, $H = 10$, $D = 20$ produit une *sphère aplatie* idéale pour un dôme souterrain ou un toit de stade. Volume ≈ 2094 blocs (33 piles). Commande WorldEdit : `//ellipsoid stone 10 5 10`.

10. Voxelisation et calcul de volume

Le volume continu de l'ellipsoïde est un résultat classique du calcul intégral, obtenu par intégration sur des disques :

$$V = \frac{4}{3} \pi r_x r_y r_z$$

La version discrète parcourt chaque voxel de la boîte $D_x \times D_y \times D_z$ et incrémente un compteur quand $a_x^2 + a_y^2 + a_z^2 \leq 1$. C'est l'algorithme `voxelVolume`.

Coque vs. intérieur. En mode *filled*, le projet n'affiche que les voxels ayant au moins un voisin-6 hors de l'ensemble préservé (voxels visibles de la surface externe ou de la face de coupe). En *thin*, seule la **surface** de l'ellipsoïde complet (épaisseur de 1 voxel) est montrée. En survie, seules les faces visibles ont besoin du bloc texturé dédié — les voxels intérieurs peuvent être un remplissage bon marché (Terre au lieu de Diamant) pour économiser des ressources.

Tableau de référence. Le tableau résume les diamètres canoniques de sphère les plus demandés par la communauté Minecraft, traduits en nombre de blocs (V), packs de 64 ($P = \lceil V/64 \rceil$) et coffres doubles de 3 456 emplacements ($DC = \lceil V/3,456 \rceil$). Sert à planifier la collecte avant de construire.

Diamètre	Blocs (V)	Packs (×64)	Coffres doubles	Note
D=8	268	5	1	petit dôme
D=12	904	15	1	sphère moyenne
D=16	2145	34	1	sommet de tour
D=20	4189	66	2	toit de stade
D=24	7238	114	3	grand dôme
D=32	17156	269	5	mégaconstruction

Volumes en mode plein calculés sous l’algorithme Euclidien pour $D \in \{8, 12, 16, 20, 24, 32\}$. Les valeurs en mode mince (coque) sont approximativement $V_{\text{coque}} \approx 4\pi r^2$ blocs d’un voxel d’épaisseur, soit entre 25% et 40% du volume plein selon D .

11. Coupes géométriques : plans et coupe diagonale à 45°

Block Round permet de couper le solide par **trois plans**. Chaque coupe est une *inégalité linéaire* qui supprime des voxels :

Axe X :
Axe Y :
Diagonale :

$x \geq cut$
 $y \geq cut$
 $x + y \geq cut$

$\Rightarrow$ supprimé
 $\Rightarrow$ supprimé
 $\Rightarrow$ supprimé

Les plans X et Y sont perpendiculaires aux axes coordonnés, avec normales $(1, 0, 0)$ et $(0, 1, 0)$. Le *plan diagonal* a pour normale $(1, 1, 0)/\sqrt{2}$ et coupe à 45° dans le plan XY : la famille $x + y = k$. Le curseur de coupe diagonale a une amplitude $D_x + D_y$, tandis que X et Y ont D_x et D_y . La coupe est **proportionnelle** : le code stocke $cutPct$ et recalcule la limite comme

$$cut = cutPct \cdot \max_{\text{axe}}$$

de sorte que 50% sur un axe reste 50% quand on change d’axe ou redimensionne.

Mapping Minecraft. La coupe Y à 50% sur une sphère produit le *dôme* classique de la moitié du volume. Pour $D = 16$: $V_{\text{full}} = 2\,145$, $V_{\text{dome}} \approx 1\,095$ blocs (≈ 18 packs). La coupe diagonale à 50% produit une section *demi-cuvette* très utilisée pour les intérieurs de fontaines et les gradins.

12. Mode Épais 3D : tridimensionnalisation par tranches

Pour le mode *thick* en 3D, le projet applique l'algorithme épais 2D sur **toutes les tranches** dans les trois axes : XY pour chaque z , XZ pour chaque y , YZ pour chaque x . Puis il fait l'*union* des résultats. La motivation est l'étanchéité : l'ensemble minimal de voxels qui bouchent toutes les diagonales sans doubler l'épaisseur de la coque. Avec $S_z(z)$ la tranche XY à la hauteur z et T l'opérateur épais 2D :

$$\text{thick3D} = \bigcup_z T(S_z(z)) \cup \bigcup_y T(S_y(y)) \cup \bigcup_x T(S_x(x))$$

Le résultat est ensuite intersecté avec l'ensemble préservé par la coupe. La théorie sous-jacente est la **topologie digitale** : l'opérateur épais garantit la *26-connexité* de la coque, utile en Minecraft où les voxels diagonaux ne se touchent pas par face — la lumière, l'eau et les créatures passent à travers.

Mapping Minecraft. Pour un dôme de Verre sous-marin sur un monument océanique en survie, le mode épais est obligatoire : les ponts diagonaux empêchent l'eau d'entrer par les fissures que le mode mince laisse ouvertes. Coût : dôme $D = 20$ en épais demande ≈ 380 blocs de Verre contre ≈ 320 en mince — 19% de matière en plus pour l'étanchéité totale.

13. Caméra 3D : coordonnées sphériques

La caméra 3D orbite l'objet à une distance r avec deux angles — θ (azimut, plan horizontal) et φ (polaire, depuis l'axe vertical). C'est la **convention physique** des coordonnées sphériques, et la conversion en cartésien (ce qu'attend *three.js*) est :

$$\begin{aligned} x &= r \sin(\varphi) \cos(\theta), \\ y &= r \cos(\varphi), \\ z &= r \sin(\varphi) \sin(\theta). \end{aligned}$$

Position initiale : $\theta = \pi/4$ (45° , perspective isométrique) et $\varphi = \pi/3$ (60° depuis le pôle Nord, légèrement au-dessus de l'équateur). Le glisser-souris incrémente

θ et φ directement ; la molette/pincette incrémente r . La simplicité de cette paramétrisation permet la rotation libre sans quaternions ni matrices explicites.

14. Auto-zoom : sphère englobante et FOV

Quand l'utilisateur change la taille de la figure ou réinitialise la caméra, le projet recalcule la **distance idéale** pour que l'objet tienne à l'écran sous tout angle. L'idée est d'englober l'objet dans une *sphère de rayon* R (la moitié de la diagonale de l'AABB) :

$$R = \sqrt{(D_x/2)^2 + (D_y/2)^2 + (D_z/2)^2}$$

Une sphère a **projection invariante par rotation**, donc si elle tient à l'écran dans une orientation, elle tient dans toutes. Étant donné le FOV vertical ($fov_V = 35^\circ$ en radians), la distance nécessaire est :

$$dist_V = \frac{R}{\tan(fov_V/2)}$$

Le FOV horizontal sort du rapport d'aspect du canevas par :

$$fov_H = 2 \arctan(\tan(fov_V/2) \cdot aspect)$$

La distance finale est le maximum entre $dist_V$ et $dist_H$, multiplié par 1,20 (20% de marge visuelle). Quand la figure est coupée, le code rétrécit D_x ou D_y à la taille restante. Quand l'easter egg du chêne ou du creeper est actif, la boîte englobante est étendue vers le haut pour inclure la hauteur de l'entité.

15. Ombrage et multiplication de texture

Les trois styles 3D (*Classic*, *Smooth*, *Blocks*) diffèrent par des **facteurs multiplicatifs** appliqués aux trois faces visibles de chaque voxel (haut, côté éclairé, côté sombre). La fonction `shadeHex(hex, factor)` parse le hex en (R, G, B) et multiplie chaque canal, avec *clamp* dans $[0, 255]$ — le résultat devient une teinte appliquée par-dessus la texture brute du bloc :

$$R' = \text{clamp}(R \cdot f), \quad G' = \text{clamp}(G \cdot f), \quad B' = \text{clamp}(B \cdot f)$$

C'est une *approximation linéaire* de la fonction d'illumination Lambertienne idéale,

$$I = \max(0, \mathbf{n} \cdot \mathbf{l}) \cdot \text{texture},$$

où les trois facteurs correspondent au cosinus de l'angle entre la normale de chaque face et une direction de lumière fixe. En *Smooth* les facteurs sont proches ; en *Classic* ils diffèrent davantage (contraste fort, le look du Minecraft vanilla). *Blocks*

introduit en plus un retrait géométrique — translation constante de chaque face vers l'intérieur, pour révéler les frontières entre voxels même quand les voisins partagent la même texture. Pour les blocs émissifs, le terme Lambertien est remplacé par une constante $I = \text{facteur_émissif} \cdot \text{texture}$.

16. Transition : des mathématiques continues à la construction in-game

Les quinze sections précédentes décrivent le pipeline *continu-vers-discret* que Block Round partage avec son projet frère Pixel Round : de l'équation implicite de l'ellipse jusqu'à la caméra qui cadre le résultat. Les six sections restantes décrivent ce qui est **spécifique** à Block Round — la couche supplémentaire de mathématiques introduite par le fait que les cellules rendues ne sont pas des pixels abstraits mais des *blocs Minecraft*, chacun avec six faces texturées, un poids dans l'inventaire et un coût en temps de collecte et de pose.

Ce ne sont pas des considérations cosmétiques. Le choix de rendre avec des textures de blocs change le problème opérationnel : au lieu d'*exporter un PNG*, l'utilisateur doit *traduire la silhouette en un plan de pose*. Le contenu mathématique des prochaines sections concerne exactement cette traduction : arithmétique d'inventaire, optimisation par comptage de faces, décomposition couche-par-couche, et exploitation de la symétrie pour réduire le coût de N poses à $N/8 + 7$ invocations de commande.

17. Arithmétique de l’inventaire Minecraft

Un joueur de Minecraft porte des items dans un *inventaire* de 36 emplacements organisés en grille 9×4 . Chaque emplacement contient jusqu’à 64 items du même type (une *pile* ou *pack*). Le joueur accède aussi au *stockage* : coffre simple 27 emplacements, coffre double 54 emplacements. L’arithmétique qui traduit un nombre cible de blocs en plan de collecte est donc un problème de *division entière par excès*.

17.1 Emplacement unique, pile unique, pack plein

La *pile* est l’unité élémentaire de comptabilité d’inventaire. Chaque pile contient au plus 64 items. La conversion du nombre N en packs est

17.2 Stockage : coffres, coffres doubles, boîtes de shulker

Pour calculer combien de *coffres doubles* une construction requiert, divisez par $54 \times 64 = 3\,456$ items par coffre double :

packs : $N_{\text{packs}} = \left\lceil \frac{N}{64} \right\rceil$ coffres doubles : $N_{\text{dc}} = \left\lceil \frac{N}{3456} \right\rceil$

Un *coffre simple* contient $27 \times 64 = 1\,728$ items. Une *boîte de shulker* (conteneur portable) contient $27 \times 64 = 1\,728$ items mais peut elle-même être rangée *dans* un emplacement de coffre, donc un coffre double rempli de 54 boîtes de shulker contient jusqu’à $54 \times 1\,728 = 93\,312$ items. Pour les mégaconstructions l’unité pertinente est le *coffre double chargé de shulkers*.

17.3 Tableau de référence : sphères canoniques

Le tableau convertit les volumes canoniques de sphère pleine en plans de collecte. La dernière colonne suggère un contexte de construction :

Diamètre	V (blocs)	Packs	Stockage	Construction typique
D=8	268	5	1 cd	toit de cabane, dôme de puits
D=12	904	15	1 cd	sommet de tour de guet
D=16	2145	34	1 cd	cloche de village, dôme de bibliothèque
D=20	4189	66	2 cd	dôme d’observatoire
D=24	7238	114	3 cd	dôme de temple
D=32	17156	269	5 cd	dôme de cathédrale, arène de boss

La représentation *pires-et-reste* $N = q \cdot 64 + r$ affichée par la puce d'info reflète la manière dont l'inventaire de survie affiche les piles partielles : q emplacements pleins plus un emplacement montrant r . Pour $D = 16$, la puce imprime « 2 145 (33×64+33) » — 33 emplacements complets plus un avec 33 items, soit exactement 34 emplacements (une ligne plus quatre).

18. Surlignage de contour et comptage des faces exposées

Block Round dessine par défaut un *contour noir* (highlight overlay) sur les arêtes de chaque face de voxel visible. Visuellement cela ressemble au style « F3+G » des frontières de chunk de l'overlay de debug, mis à l'échelle voxel. Mathématiquement, l'overlay implémente la visualisation du **comptage des faces exposées** : un nombre que les bâtisseurs de survie utilisent pour valider la progression bloc par bloc.

18.1 Qu'est-ce qu'une face exposée

Une face du voxel $\mathbf{v}$ partagée avec le voisin $\mathbf{n}$ est *exposée* si et seulement si $\mathbf{n} \notin V_{\text{solide}}$ (le voisin est vide). Chaque voxel solide contribue entre 0 (entièrement enterré) et 6 (isolé) faces. Le total F_{exp} est l'analogie discret de l'*aire de surface*.

18.2 Formule de détection de frontière

Un voxel est sur la frontière si au moins un de ses six voisins-face est hors du solide :

$$b(i, j, k) = f(i, j, k) \wedge (\neg f(i \pm 1, j, k) \vee \neg f(i, j \pm 1, k) \vee \neg f(i, j, k \pm 1))$$

C'est le même opérateur de frontière que l'algorithme mince 2D étendu en 3D, et l'overlay dessine simplement les arêtes cubiques de chaque voxel-frontière. Pour Verre et Glace, l'overlay est OFF par défaut ; pour Pierre, Pierres Moussues, Diamant et autres blocs opaques, il est ON, car l'overlay distingue les voxels internes de la coque.

18.3 Comptage de faces exposées comme heuristique de survie

Pour les bâtisseurs en survie, le nombre pertinent n'est pas le volume V mais les *faces exposées* F_{exp} , car c'est ce qui est visible de l'extérieur. La somme totale égale six fois le nombre de voxels-frontière moins le double des adjacences-face

entre voxels-frontière :

$$F_{\text{exp}} = \sum_{\mathbf{v} \in V_{\text{solid}}} \sum_{\mathbf{n} \in N_6(\mathbf{v})} [\mathbf{v} + \mathbf{n} \notin V_{\text{solid}}]$$

Pour une sphère $D = 16$ (mode plein), $V = 2\,145$, $V_{\text{frontière}} \approx 432$, $F_{\text{exp}} \approx 1\,184$ faces visibles — la construction n’a besoin que d’environ 432 blocs *visibles*; les 1 713 restants peuvent être remplis du bloc le moins cher (Terre, Pierres Moussues). L’overlay permet de repérer un bloc manquant dans la coque immédiatement.

19. Construction couche-par-couche (décomposition horizontale)

En Minecraft, la construction progresse *par couches horizontales* : le joueur est sur une couche, pose les blocs de la couche supérieure, puis monte d’un bloc. Le problème 3D de l’ellipsoïde se comprend donc mieux comme une *pile d’ellipses 2D*, une par couche :

$$\text{couche}(k) = \left\{ (i, j) : a_x(i)^2 + a_y(j)^2 \leq 1 - a_z(k)^2 \right\}$$

Pour chaque couche entière k de $-r_z$ à $+r_z$, la section transverse est elle-même une ellipse de demi-axes rétrécis $r_x(k)$ et $r_y(k)$:

$$r_x(k) = r_x \sqrt{1 - \left(\frac{k}{r_z}\right)^2}, \quad r_y(k) = r_y \sqrt{1 - \left(\frac{k}{r_z}\right)^2}$$

Cela réduit le problème 3D à *calculer l’algorithme 2D des sections précédentes, une couche à la fois*, et empiler les résultats. Cognitivement c’est une énorme simplification : au lieu de trois coordonnées mentales, le bâtisseur en suit deux par couche plus l’indice de couche.

Mapping Minecraft. Pour une sphère $D = 16$ ($r_z = 8$), la couche équatoriale ($k = 0$) est le cercle Bresenham $D = 16$ complet (≈ 200 blocs). La suivante ($k = 1$) est un cercle $D = 15,97$. En $k = 4$ c’est un cercle $D = 13,86$ (≈ 148 blocs), et en $k = 7$ un $D = 7,48$ (≈ 43 blocs). Les pôles ($k = \pm 8$) sont une calotte d’un bloc.

La puce d’info de Block Round expose ce comptage par couche en mode 3D quand l’utilisateur active le bouton Guides centraux (touche c). Les tranches horizontales sont dessinées comme une croix jaune translucide à l’équateur et à chaque subdivision $r_z/2$.

20. Optimisation par symétrie octaédrique (O_h)

Une sphère centrée à l'origine est invariante sous l'action du **groupe octaédrique** O_h d'ordre 48. La voxelisation préserve un sous-groupe d'ordre 48 engendré par les trois réflexions de coordonnées et les trois rotations de $\pi/2$ autour de chaque axe. En pratique de construction, le sous-groupe pertinent est le groupe de réflexions $\mathbb{Z}_2^3$ d'ordre 8, engendré par $x \mapsto -x$, $y \mapsto -y$, $z \mapsto -z$. L'ensemble de voxels dans un octant détermine la sphère entière :

$$V_{\text{total}} \approx 8 V_{\text{octant}} \implies \text{réduction} = \frac{N - N/8}{N} = 87,5\%$$

C'est la plus grande optimisation mathématique disponible pour un bâtisseur Minecraft. Au lieu de poser manuellement N blocs, le bâtisseur en pose $N/8$ dans l'octant positif puis émet sept commandes `/clone` (vanilla) ou `//copy + //paste` (WorldEdit) avec les réflexions appropriées.

20.1 Séquence de commandes concrète

Pour une sphère $D = 24$ ($r = 12$, $V = 7\,238$ blocs) centrée en $(0, 72, 0)$, construisez d'abord l'octant positif (région $+x, +y, +z$, ≈ 905 blocs), puis émettez :

```
/clone 0 72 0 12 84 12 ~~~ filtered minecraft:stone
/clone 0 72 0 12 84 12 -12 72 0 mode:masked (réflex. -x)
/clone 0 72 0 12 84 12 0 60 0 mode:masked (réflex. -y)
/clone 0 72 0 12 84 12 0 72 -12 mode:masked (réflex. -z)
/clone -12 72 0 -1 84 12 mode:masked (octant -x, -y)
/clone 0 60 -12 12 71 -1 mode:masked (octant -y, -z)
/clone -12 72 -12 -1 84 -1 mode:masked (octant -x, -z)
/clone -12 60 -12 -1 71 -1 mode:masked (octant -x, -y, -z)
```

Chaque `/clone` coûte environ 50 ms de temps serveur, donc les sept réflexions finissent en moins d'une demi-seconde. La pose manuelle des 905 blocs de l'octant prend environ 15 min en survie ou 45 s en créatif au pinceau. La pose complète des 7 238 blocs sans symétrie prendrait ≈ 2 h en survie ou 6 min en créatif.

20.2 Comparaison de temps

Soit T_{block} le temps de pose par bloc (typiquement 1–2 s en survie, 0,1–0,5 s en créatif) et T_{clone} le temps par commande clone (≈ 50 ms). Le temps total avec et sans symétrie est :

$$T_{\text{octant}} + 7 T_{\text{clone}} \ll T_{\text{full}}$$

Pour $N = 7\,238$ en survie ($T_{\text{block}} = 2\text{ s}$, $T_{\text{clone}} = 0,05\text{ s}$) : complet = $14\,476\text{ s} \approx 4\text{ h } 1\text{ min}$; octant + clones = $1\,810\text{ s} + 0,35\text{ s} \approx 30\text{ min}$. Un facteur 8 qui correspond exactement à l'ordre du sous-groupe de réflexions. Le O_h complet d'ordre 48 pourrait théoriquement réduire encore (facteur 48), mais exploite des rotations diagonales nécessitant une zone tournée de 45° , rarement utile en Minecraft vanilla.

21. Textures de blocs et composition visuelle

Un bloc Minecraft n'est pas une cellule plane colorée mais un *cube avec six faces texturées*. Block Round rend chaque voxel avec ses six textures canoniques de face, échantillonnées du resource pack vanilla. Mathématiquement, la texturation est un *mappage UV* : chaque face du cube unité est paramétrée par des coordonnées $[0, 1]^2$, et la texture correspondante est échantillonnée à ces UVs.

21.1 Les six faces et le mappage UV

Pour chaque face de voxel le moteur consulte le modèle du bloc :

$$\text{face} \in \{\text{haut, côté, bas}\} \quad \text{UV} : [0, 1]^2 \rightarrow \text{pixels}$$

La plupart des blocs (Pierre, Pierres Moussues, Bloc de Diamant, Planches de Chêne) utilisent la *même texture sur les six faces*. Les blocs multi-faces (Bloc d'Herbe, Citrouille, Botte de Foin, Pilier de Quartz, tous les rondins, Établi, Fourneau, Bibliothèque, TNT) utilisent des *textures différentes* en haut, côté et bas : Bloc d'Herbe a un haut vert, un côté herbe-et-terre et un bas de terre ; un Rondin de Chêne a l'anneau du bois en haut/bas et l'écorce sur les quatre côtés. Crucialement, la *mathématique de quels voxels poser* ne dépend pas de la texture — la silhouette d'une sphère de Diamant et celle d'une sphère de Pierres Moussues du même diamètre est identique. La décision de texture est une couche *purement esthétique* appliquée après l'algorithme géométrique.

21.2 Familles de blocs et tons

Pour le bâtisseur de survie choisissant parmi les dizaines de blocs disponibles, la question pertinente n'est pas l'indice de texture mais la *famille tonale* : clair/sombre, chaud/froid, uniforme/à motif. Le tableau classe les blocs les plus utilisés en constructions Block Round :

Bloc	Famille	Ton	Usage typique
cobblestone	pierre	gris	murs rustiques, donjons
stone	pierre	gris	murs propres, socles
oak_planks	bois	beige	sols, finitions intérieures
quartz_block	quartz	blanc	temples, dômes modernes
sandstone	sable	beige	déserts, pyramides
deepslate	pierre	sombre	souterrains, accents

21.3 Dégradés par mélange de blocs

Bien qu'un seul type de bloc fournisse un ton, des *dégradés* s'obtiennent en alternant deux ou trois types par couche. Une technique classique est le dégradé *pierre-vers-quartz* sur un dôme : couches inférieures Pierre Lisse, couches équatoriales un mélange bruité de Pierre Lisse et Diorite, couches supérieures Quartz. Chaque couche reste calculée par le même algorithme Block Round ; ce qui change est la consultation de texture par voxel. L'option de bloc *Random* de Block Round implémente un tel mélange procédural par défaut (sommet d'herbe, terre dessous, minerais épars, deepslate près du bedrock). La mathématique est celle de la décomposition par couche de la Section 19 ; seule l'« étiquette de bloc » par voxel change.

22. Conclusion

Ce document a décrit, de manière systématique, les fondements mathématiques employés dans Block Round. En environ mille lignes de JavaScript plus les pipelines de texture et d'audio, le projet intègre des contributions de plusieurs domaines théoriques :

- **Géométrie analytique** : équations implicites de l'ellipse et de l'ellipsoïde comme critère primaire d'appartenance.
- **Algorithmes classiques de rasterisation** : Bresenham en formulation de point milieu, avec extension de Pitteway/Kappel pour les ellipses.
- **Topologie digitale** : opérateurs de frontière discrète, concepts de 4-, 6- et 26-connexité, composition par tranches et comptage des faces exposées.
- **Calcul intégral** : volume de l'ellipsoïde comme intégrale triple et son équivalent discret (mesure de comptage sur voxels).
- **Algèbre linéaire** : plans de coupe définis par des inégalités linéaires et pro-

jection perspective tridimensionnelle.

- **Trigonométrie** : paramétrisation de la caméra en coordonnées sphériques et ajustement automatique de distance en fonction du FOV.
- **Arithmétique combinatoire d'inventaire** : comptabilité par division entière par excès des packs de 64 et des coffres doubles de 3 456 items, avec la représentation piles-et-reste affichée par la puce d'info.
- **Symétrie de groupes** : exploitation du groupe octaédrique O_h et de son sous-groupe de réflexions d'ordre 8 $\mathbb{Z}_2^3$ pour réduire le coût de construction d'un facteur 8 via `/clone`.

L'observation centrale que l'étude permet de formuler est la suivante : sur une grille discrète, **il n'existe pas de représentation unique correcte** d'une courbe continue. Chaque algorithme présenté dans ce document correspond à une définition mathématique distincte du critère d'inclusion des cellules, et chaque définition produit une silhouette aux propriétés formelles propres — douceur dans Euclidien, motif en escalier de Bresenham, couverture plus large dans la couverture-par-coin. Le choix de l'algorithme est donc une décision technique qui doit considérer la représentation visuelle souhaitée, les caractéristiques métriques visées et — spécifiquement pour Block Round — le coût en inventaire et en temps de construire physiquement la forme dans le jeu.

Block Round occupe une niche petite mais mathématiquement riche dans l'écosystème d'outils Minecraft : il se situe entre les outils purement géométriques (WorldEdit, Litematica) et les outils purement visuels (resource packs, shaders), offrant un pont explicite, dérivable et reproductible de l'équation continue au plan de pose discret. Le projet frère Pixel Round offre les mêmes algorithmes sans la couche Minecraft, adapté au pixel art et au voxel art traditionnels. Ensemble les deux projets illustrent comment le même pipeline mathématique continu-vers-discret soutient des flux artistiques et constructifs radicalement différents.

Annexe A : tableaux de référence de construction et exemple détaillé

Cette annexe rassemble les références numériques et de commandes les plus utilisées par les utilisateurs de Block Round. Les données consolident les tableaux par section et les exemples pratiques disséminés dans le document, organisés pour servir de consultation d’une page pour la planification, la syntaxe des commandes et un exemple détaillé de bout en bout.

A.1 Référence complète des sphères (pleine, creuse, packs)

Le tableau étend les tables canoniques des Sections 10 et 17 en ajoutant la variante creuse (mode *mince*) à côté du volume plein. Les comptes de coque creuse supposent un seul voxel d’épaisseur et sont calculés sous l’algorithme Euclidien ; le mode épais ajoute environ 10–15% de blocs supplémentaires pour sceller les fissures diagonales selon la Section 12.

D	Pleine (V)	Creuse (coque)	Packs pleine	Packs creuse
8	268	170	5	3
10	523	316	9	5
12	904	500	15	8
14	1437	744	23	12
16	2145	1042	34	17
18	3052	1338	48	21
20	4189	1648	66	26
24	7238	2400	114	38
28	11494	3296	180	52
32	17156	4332	269	68

Pour un bâtisseur en survie, la colonne *creuse* est généralement la pertinente : seule la coque visible a besoin du bloc texturé dédié, tandis que l’intérieur (si désiré) peut être rempli du matériau le moins cher. La colonne packs-pleine donne le stock total pour une sphère entièrement remplie ; packs-creuse est le minimum réaliste pour un dôme creux.

A.2 Référence de commandes WorldEdit / vanilla

Le tableau résume les commandes WorldEdit et Minecraft vanilla les plus pertinentes pour construire les sorties de Block Round in-game. Les commandes

WorldEdit supposent que le joueur tient une baguette (hache en bois par défaut) et a sélectionné une région ; les commandes vanilla fonctionnent sans mod mais demandent des arguments de coordonnées.

Commande	Mod / origine	Résultat
<code>//sphere stone 8</code>	WorldEdit	sphère pleine $r=8$ (Euclidien)
<code>//hsphere stone 8</code>	WorldEdit	sphère creuse $r=8$ (coque seule)
<code>//cyl stone 8 16</code>	WorldEdit	cylindre $r=8$, $h=16$
<code>//ellipsoid st. 10 5 10</code>	WorldEdit	ellipsoïde $10 \times 5 \times 10$
<code>/fill ... stone</code>	vanilla	remplir boîte d'un seul bloc
<code>/clone ... mode:masked</code>	vanilla	copier région vers nouvel emplacement

Pour les utilisateurs de Litematica, le flux est différent : au lieu d'exécuter des commandes, le fichier schématique (Block Round exporte `.schem`, format Sponge v2) est chargé dans le mod Litematica, qui affiche un fantôme translucide de la figure que le joueur place bloc par bloc. C'est l'équivalent survie le plus proche de l'expérience créative de WorldEdit.

A.3 Exemple détaillé : dôme $D=20$ de Quartz sur un temple

Comme exemple détaillé complet, supposez qu'un bâtisseur veuille un dôme de Quartz $D = 20$ au sommet d'un temple de pierre existant. Le sommet du temple fait 11×11 blocs, centré en $(0, 80, 0)$. Le dôme est la moitié supérieure d'une sphère de rayon $r = 10$ coupée au plan $Y = 0$. Le plan est :

1. **Planifier avec Block Round.** Ouvrir l'app, mode 3D, Sphère, taille $D = 20$, axe de coupe Y à 50%. Sélectionner Bloc de Quartz. La puce d'info rapporte $V_{\text{dôme}} \approx 2\,126$ blocs (≈ 34 packs, 1 coffre double avec 1 330 items restants dans le second).
2. **Rassembler les ressources.** 34 packs de Bloc de Quartz $= \lceil 2\,126/4 \rceil = 532$ Blocs de Quartz $\times 4$ Quartz du Nether chacun $= 2\,128$ Quartz du Nether. Fondre ou échanger.
3. **Choisir l'algorithme.** Pour un dôme vu d'en bas, l'algorithme Euclidien donne la silhouette la plus lisse ; pour un dôme sur une montagne distante, Seuil se lit plus net. Block Round utilise Euclidien par défaut.
4. **Positionner le dôme.** La face plate doit coïncider avec le sommet du temple. La boîte englobante est $20 \times 10 \times 20$ blocs ; placer son centre en $(0, 80, 0)$, la face plate sera en $y = 80$ et l'apex en $y = 89$.

5. **Construire par symétrie.** Construire le quadrant $+x, +z$ (≈ 530 blocs). Émettre `/clone 0 80 0 10 89 10 -10 80 0 mode:masked` pour refléter en x ; puis `/clone -10 80 0 10 89 10 0 80 -10 mode:masked` pour refléter en z . Temps total : ~ 10 minutes en survie.
6. **Polir.** Ajouter des Lanternes Marines à l’apex et aux quatre points cardinaux de l’équateur pour l’éclairage émissif (Block Round les rend avec leur carte émissive au niveau de lumière MC 15). Optionnellement basculer le surlignage de bord (touche **G**) pour vérifier la coque avant de poser les derniers blocs.

Les mathématiques de chaque étape sont exactement ce que les vingt-deux sections précédentes décrivent : équation implicite de l’ellipsoïde, voxelisation Euclidienne, coupe en Y, réduction par symétrie octaédrique via `/clone`, arithmétique d’inventaire pour traduire le volume en packs et coffres. L’annexe n’est donc pas une théorie nouvelle — c’est un rappel que chaque concept théorique du document se traduit en une décision concrète unique dans le flux de planification.

A.4 Référence des raccourcis clavier

L’app Block Round expose un petit ensemble de raccourcis clavier qui reflètent les boutons d’angle sur le canevas. Ils sont listés ci-dessous pour référence rapide ; les raccourcis sont globaux (aucun champ de saisie n’a besoin du focus) et fonctionnent en modes 2D et 3D sauf indication contraire.

Touche	Bascule	Notes
G	Overlay de bord	contour noir sur chaque face de voxel
C	Guides centraux	croix jaune translucide sur les axes
D	Télécharger PNG	canevas actuel comme image PNG
I	Puce d’info	compte de blocs et détail des packs
M	Bascule 2D / 3D	passé entre mode plat et voxel
S	Son on/off	sons ambiants de pose
T	Thème jour / nuit	assombrit le fond, garde les tuiles lisibles
Ctrl+Z / Y	Annuler / refaire	parcourt l’historique de modifications de la figure

Les raccourcis **G** (grille) et **C** (guides centraux) sont particulièrement utiles pendant la validation de la construction : **G** montre si chaque voxel de la coque est en place (un bloc manquant brise le motif de l’overlay), et **C** confirme que les coupes sont bien centrées sur les axes de la figure.

Pour le **Ctrl+Z** d’annulation : Block Round parcourt chaque édition qui change la figure (curseurs, mode de rendu, algorithme, forme, axe, bloc, bascule de mode)

mais exclut délibérément les bascules purement visuelles (caméra, overlay de bords, croix centrale, thème, son). Cette séparation maintient la pile d'undo peuplée d'édicions que l'utilisateur voudra probablement annuler.

A.5 Glossaire des termes clés

Référence rapide aux termes techniques qui se répètent dans ce document. Chaque entrée récapitule brièvement le concept et pointe vers la section où il est développé en détail.

- **Voxel** — cube unité en 3D, analogue du pixel en 2D. Chaque bloc Minecraft est un voxel. Voir section 2.
- **Pack (pile)** — en Minecraft, une pile de 64 items identiques occupant un emplacement d'inventaire. Voir section 17.
- **Coffre double** — deux coffres adjacents fusionnés en un conteneur de 54 emplacements. Contient jusqu'à 3 456 items. Voir section 17.
- **Boîte de shulker** — conteneur portable de 27 emplacements pouvant être lui-même stocké dans un emplacement de coffre. Voir section 17.
- **Coque** — la couche externe d'un voxel d'épaisseur d'une figure solide. La surface pertinente pour les bâtisseurs en survie. Voir sections 7 et 18.
- **Octant** — une des huit régions de l'espace 3D obtenues par intersection des trois demi-espaces coordonnés. Utilisée dans l'optimisation par symétrie. Voir section 20.
- **Mappage UV** — une paramétrisation 2D d'une face 3D, utilisée pour mapper une image de texture sur chaque face d'un voxel. Voir section 21.
- **Bresenham** — algorithme incrémental de rastérisation de 1965 utilisant uniquement de l'arithmétique entière. Voir section 5.
- **Seuil** — critère de rastérisation incluant une cellule si l'un de ses coins est à l'intérieur de l'ellipse. Voir section 6.
- **Euclidien** — critère de rastérisation incluant une cellule si son centre est à l'intérieur de l'ellipse. Voir section 4.
- **Easter egg** — fonctionnalité cachée déclenchée par une entrée spécifique. Dans Block Round : le chêne et le creeper, tous deux à la valeur 15 du curseur.
- **Schématique** — fichier de structure sérialisé (Sponge Schematic v2, `.schem`)

chargeable par WorldEdit, Litematica ou MCEdit.

Ce glossaire reste délibérément plus court qu'un lexique mathématique complet. Les lecteurs cherchant une couverture plus profonde des mathématiques sous-jacentes (topologie digitale, calcul intégral, théorie des groupes, etc.) sont renvoyés aux sections pertinentes de ce document.

A.6 Perspectives et limites du modèle

Le modèle Block Round présenté dans ce document couvre la voxelisation statique des ellipses, ellipsoïdes et leurs coupes plus les considérations pratiques introduites par la couche Minecraft. Il ne traite pas les problèmes dynamiques — figures en mouvement, transitions de voxels animées, interactions avec simulation de fluides — car ils nécessitent un cadre temporel continu hors du périmètre de la rasterisation discrète. Une extension naturelle est l'*animation au niveau de schématique* : une séquence de voxelisations statiques reliées par interpolation, que Block Round peut déjà exporter comme un flux `.schem` lisible par des scripts Litematica.

Une autre direction ouverte est la *voxelisation exacte préservant le volume* : le comptage discret actuel V_{disc} converge vers le volume continu V_{cont} seulement asymptotiquement. Pour des applications où le comptage exact doit correspondre à la formule continue (p.ex. compétitions de rendu, art mathématique avec budgets de blocs stricts), une couche supplémentaire de raffinement est nécessaire — par exemple, en ajustant légèrement le rayon pour que le comptage discret atteigne la valeur cible. Block Round n'implémente pas actuellement ce raffinement, mais les mathématiques sont directes : résoudre $V_{\text{disc}}(r) = V_{\text{cible}}$ numériquement pour r .

Enfin, l'*exploitation de symétrie d'ordre supérieur* : le présent document couvre le sous-groupe de réflexions d'ordre 8 de O_h , mais le groupe complet d'ordre 48 inclut des rotations qui permettraient en principe une accélération de $\times 48$. L'obstacle pratique est que les commandes in-game opèrent sur des régions alignées sur les axes, et exploiter les symétries rotationnelles nécessite une zone de construction tournée à 45° . Les mods serveur qui font tourner les boîtes de sélection en interne (p.ex. FAWE) peuvent en principe y parvenir, mais le tooling courant ne l'expose pas encore. Une implémentation de référence démontrant un facteur $\times 48$ sur un serveur vanilla serait un suivi naturel au présent travail.

A.7 Références supplémentaires et ressources externes

Les concepts mathématiques développés dans ce document ont des littératures bien établies. Pour les lecteurs cherchant des sources primaires ou un approfondissement, voici quelques références :

dissement algorithmique, les points de départ suivants sont recommandés :

- J. E. Bresenham, *Algorithm for computer control of a digital plotter*, IBM Systems Journal, vol. 4 no. 1, 1965 — l'article original de rasterisation par arithmétique entière étendu dans ce document.
- M. L. V. Pitteway, *Algorithm for drawing ellipses or hyperbolae with a digital plotter*, Computer Journal, vol. 10, 1967 — généralise Bresenham aux coniques arbitraires.
- A. Kappel, *An ellipse-drawing algorithm for raster displays*, ACM Comm. vol. 28, 1985 — la formulation elliptique à deux régions utilisée dans la section 5.
- R. Klette et A. Rosenfeld, *Digital Geometry : Geometric Methods for Digital Picture Analysis*, Morgan Kaufmann, 2004 — référence standard pour la topologie digitale et les opérateurs de frontière des sections 7 et 18.
- Documentation WorldEdit, enginehub.org/worldedit/ — référence officielle des commandes `//sphere`, `//hsphere`, `//ellipsoid`, `//copy` utilisées tout au long de ce document.
- Spécification Sponge Schematic v2, github.com/SpongePowered/Schematic-Specification — le format de fichier que Block Round exporte comme `.schem`.

Les huit domaines théoriques listés à la section 22 ont chacun des traitements au niveau du manuel qui vont bien au-delà de ce qu'un seul document technique peut résumer. La contribution de Block Round n'est pas dans la théorie elle-même mais dans la synthèse spécifique : appliquer la rasterisation classique au cadre Minecraft de voxels texturés, avec les considérations explicites d'inventaire et de coût de symétrie habituellement omises des outils purement géométriques.

Une dernière remarque sur la reproductibilité : tout le pipeline de ce document — script de build, traductions, modèle LaTeX, tableaux par locale — est contenu dans le répertoire `docs_math/` du dépôt du projet. Réexécuter `python build.py` sur un système avec XeLaTeX/MiKTeX produit ce PDF exactement, dans n'importe lequel des neuf locaux pris en charge. Le projet frère Pixel Round suit la même structure, permettant une comparaison directe côte à côte des variantes texturée et non texturée.

Fin du document

Block Round est maintenu par Vinícius Rodrigues de Souza. Le projet frère Pixel

Round est disponible à github.com/ViniSouza128/pixel-round; le dépôt actuel de Block Round est à github.com/ViniSouza128/block-round.

Commentaires, corrections et améliorations de traduction sont bienvenus via le suivi de problèmes du dépôt du projet. La relecture par des locuteurs natifs des huit locales non anglaises est particulièrement appréciée, car les localisations ont été préparées avec une révision substantielle mais la terminologie technico-mathématique bénéficie d'une correction native.